Operating Systems: Interprocess Communication (IPC) & Distributed Foundations

Pipes, UNIX Domain Sockets, POSIX Shared Memory, Message Queues, and RPC Mechanisms

Doc Id: OS-IPC-2024-006 | **Practice:** IPC, Distributed OS & Sockets | **System:** Linux POSIX IPC, gRPC / Protocol Buffers, UNIX Domain Sockets | **Year:** 2024

1. IPC Taxonomy and Communication Models

Interprocess Communication (IPC) enables isolated processes with independent virtual address spaces to exchange data and synchronize actions.

IPC models divide into two fundamental architectures:

1. **Shared Memory Architecture:** Processes map the identical physical memory frames into their respective virtual address spaces. Once established, data transfers occur at raw memory bus speed without kernel intervention. Synchronization via mutexes or semaphores is strictly required.

2. **Message Passing Architecture:** Data is encapsulated in explicit message payloads transferred through kernel-managed buffers via system calls. Eliminates user synchronization requirements, but incurs double-copy kernel overhead.

IPC Mechanism	Data Paradigm	Cross-Host?	Kernel Involvement on Transfer	Benchmark Bandwidth / Latency
Anonymous Pipe	Byte stream (FIFO)	No	High (kernel buffer copy)	~2.5 GB/s | 1.8 µs
Named Pipe (FIFO)	Byte stream (filesystem node)	No	High (kernel buffer copy)	~2.4 GB/s | 2.0 µs
POSIX Message Queue	Discrete structured messages	No	High (kernel queue management)	~1.8 GB/s | 2.4 µs
POSIX Shared Memory	Raw shared address space	No	Zero (direct DRAM read/write)	~35.0 GB/s | 0.08 µs
UNIX Domain Socket	Stream or Datagram	No	Moderate (socket buffer copy)	~4.2 GB/s | 1.2 µs
TCP / UDP Sockets	Network packet stream	Yes	High (TCP/IP network stack)	~1.1 GB/s | 25 - 45 µs
gRPC / Remote Call (RPC)	Typed protobuf serialized RPC	Yes	High (serialization + network)	~0.4 GB/s | 120 - 250 µs

2. Detailed Analysis of Pipes and Sockets

Anonymous Pipes: Created via pipe(int fd[2]), returning a read descriptor fd[0] and write descriptor fd[1]. Unidirectional communication strictly between related processes sharing parent-child ancestry. Bounded kernel buffer (typically 64 KB on Linux); write blocks if pipe buffer is full, read blocks if pipe buffer is empty.

UNIX Domain Sockets (AF_UNIX): Bidirectional stream (SOCK_STREAM) or datagram (SOCK_DGRAM) endpoints addressed by filesystem pathnames. Bypasses all IP checksumming, routing, and packet encapsulation logic of TCP/IP, delivering over 3x the throughput of loopback TCP (127.0.0.1) with support for passing open file descriptors across process boundaries via SCM_RIGHTS.

3. Remote Procedure Call (RPC) & Distributed Primitives

Remote Procedure Calls (RPC) abstract network communication so that invoking a procedure on a remote server has the syntactic interface of a local subroutine call.

RPC Execution Flow:

1. Client code calls local **Client Stub** with standard function arguments.
2. Client Stub marshals (serializes) parameters into a network-neutral canonical format (Protocol Buffers, JSON, XDR).
3. Client Stub invokes OS transport layer to transmit message packet over TCP/UDP to remote server.
4. Remote OS delivers packet to **Server Stub**, which unmarshals parameters and invokes the actual server procedure.
5. Server execution produces return values; Server Stub marshals results and sends network reply packet.
6. Client Stub receives reply, unmarshals return value, and returns to client caller transparently.

> *Distributed Failure Semantics: Unlike local function calls which guarantee exactly-once execution, distributed RPCs operate over unreliable networks and must handle dropped packets and crashes using: At-Least-Once (retry on timeout), At-Most-Once (idempotency tokens / deduplication), or Exactly-Once (transactional distributed logging).*